

Jenkins File creation for Java and Python

Class no:2

Jenkinsfile

Jenkinsfile starts with the **pipeline**. The pipeline starts with the **agent**. Inside the agent we have multiple **stages** such as development, test, pre-production, production or more.

Each stage has multiple **steps** inside it.

Hands on : Creating a java item using pipeline:

Step no:1 Here we are going to create an executable from java source code.

Go to Jenkins → new item → Name: **Java-Project** → Pipeline → Ok.

New Item

Enter an item name

Java-project

Select an item type



Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

Step no:2 Configure page will open → scroll down → Definition: **Pipeline Script from SCM** → choose **Git**.

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

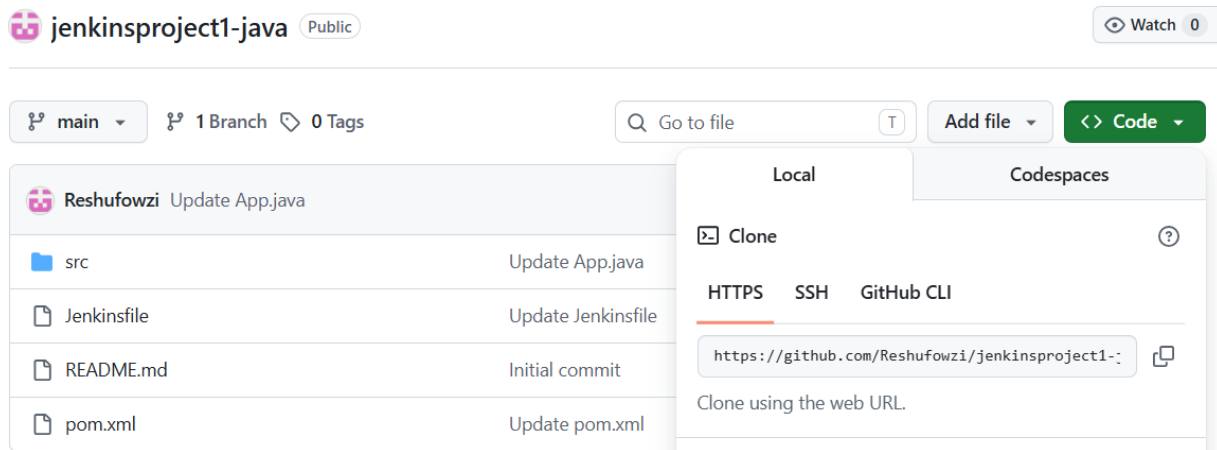
Repository URL ?

https://github.com/Reshufowzi/jenkinsproject1-java.git

Credentials ?

- none -

It will ask for repo url. Go to github and get the url of the java project.



Ensure we are entering the correct branch of the jenkinsfile and the project source code.

In script path enter the correct path of the Jenkinsfile in the repo. Globally it is advised to use Jenkinsfile as the name.

Best practice is to keep the file in the root level. If it is inside, we have to provide the absolute path, which is kind of complex.

Branch Specifier (blank for 'any') ?

*/main

Script Path ?

Jenkinsfile

Step no:3 The Jenkinsfile for our Java Project.

```
pipeline {  
    agent any           // it starts with the agent  
  
    tools {              // java requires maven to execute and we have specified the java version needed  
        maven 'Maven-3.9'  
        jdk 'JDK-17'  
    }  
  
    stages {             // as we know there are multiple stages inside the Jenkins file. It is Build stage here.
```

```

stage('Build') {
    steps {
        sh 'mvn clean compile'           // each stage has multiple steps. First we will compile
    }
}

stage('Test') {
    steps {
        sh 'mvn test'                   // followed by test of maven
    }
}

stage('Package') {
    steps {
        sh 'mvn package'                // make it as a package
    }
}

}

post {

    success {
        echo 'Build Successful!'         // post is optional. This is to expose the execution is successful or not in the
                                          console output
    }
    failure {
        echo 'Build Failed!'
    }
}

}

```

Step no:4 Now we need to set up the tools for the java project execution. We can set them up in the **manage Jenkins** Click on the settings icon.



 Add description

Choose **tools**

System Configuration



System

Configure global settings and paths.



Tools

Configure tools, their locations and automatic installers.



Nodes

Add, remove, control and monitor the various nodes that Jenkins runs jobs on.



Clouds

Add, remove, and configure cloud instances to provision agents on-demand.

Check the name of the tools we required that we mentioned in the Jenkinsfile.

```
tools {  
  maven 'Maven-3.9'  
  jdk 'JDK-17'  
}
```

Under the tools choose Add **JDK** , add the version needed.

JDK installations

+ Add JDK

JDK

Name

JDK-17

JAVA HOME

Scroll down , we have **maven** option add the version required.

Maven

Name

Maven-3.9

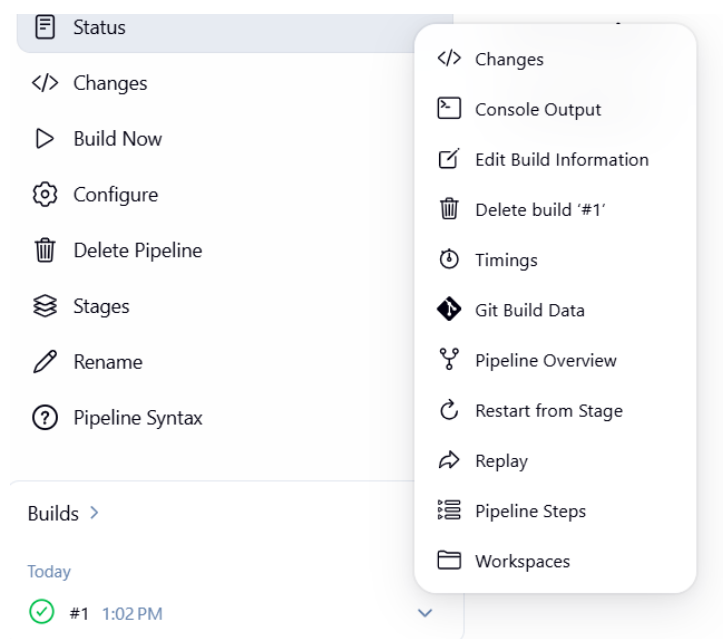
☒ Install automatically ?

Install from Apache

Version

3.9.16

Step no:5 Now go the Jenkins project page → Choose **build now** → once the build has been executed → Go to the **console output**.



We can see the build id success in the console. Because we use the post command to see the output.

Unlike the docker, where we can see the output by coping the public ip , here we can only see the output via the console output.

Now the java source code has been successfully converted into an executable Jar file.

In the console output itself we can see the location of the jar file.

```
Downloaded from central: https://repo.maven.apache.org/maven2/com/github/luben/zstd-jni/1.5.7-6/zstd-jni-1.5.7-6.jar (7.4 MB at 18 MB/s)
[INFO] Building jar: /var/lib/jenkins/workspace/Java-project/target/jenkinsproject1-java-1.0-SNAPSHOT.jar
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.395 s
[INFO] Finished at: 2026-06-03T13:02:45Z
[INFO] -----
```

Copy the path and paste it in the Ec2 console of our Jenkins, we can see the file inside it.

```
ubuntu@ip-172-31-14-119:~$ cd /var/lib/jenkins/workspace/Java-project/target/
ubuntu@ip-172-31-14-119:/var/lib/jenkins/workspace/Java-project/target$ ls
classes generated-sources generated-test-sources jenkinsproject1-java-1.0-SNAPSHOT.jar
ubuntu@ip-172-31-14-119:/var/lib/jenkins/workspace/Java-project/target$
```

Hands on: Building a jenkinsfile for a python project and pushing the executable to the dockerhub

Step no:1 Each program requires different prerequisites. But they follow the same order : agent → stages → steps.

Here we have the python project Jenkins file.

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = "python-project"                // it requires environment
        DOCKER_HUB = "nithya65"                      // I am providing the docker hub account where it should push
        BUILD_TAG = "${BUILD_NUMBER}"                // we are adding the build number of the jenkinsfile as the tag to
    }                                                the docker image

    stages {

        stage('Install & Test') {
            steps {
                sh '''
                    python3 -m venv venv
                    . venv/bin/activate
                    pip install --upgrade pip                // like maven for java, we need pip for python
                    pip install -r app/requirements.txt      // it is installed under the requirements.txt dir with
                    # Fix module path issue                  all other dependencies needed.
                    export PYTHONPATH=$PWD
                    pytest tests/
                    '''
            }
        }

        stage('Build & Push Docker Image') {
            steps {
                withCredentials([usernamePassword(          // we are going to push the build as an image to Dockerhub
                    credentialsId: 'docker-creds',         // Here were providing the Id for the Docker hub credentials
                    usernameVariable: 'USER',

```

```

        passwordVariable: 'PASS'
    )) {

        sh ""

        echo "$PASS" | docker login -u "$USER" --password-stdin

        docker build -t $DOCKER_HUB/$IMAGE_NAME:$BUILD_TAG app/           // build of the image
        docker tag $DOCKER_HUB/$IMAGE_NAME:$BUILD_TAG                     // tag the image
$DOCKER_HUB/$IMAGE_NAME:latest

        docker push $DOCKER_HUB/$IMAGE_NAME:$BUILD_TAG                   // push the image
        docker push $DOCKER_HUB/$IMAGE_NAME:latest

        ""

    }
}

stage('Deploy') {
    steps {
        echo " 🚀 Deploy step (optional)"
    }
}

post {
    success {
        echo " ✅ Success: $DOCKER_HUB/$IMAGE_NAME:$BUILD_TAG"
    }
    failure {
        echo " ❌ Failed"
    }
}
}

```

Step no:2 Here we are going to create an image from Python code.

Go to Jenkins → new item → Name: **Python-Project** → Pipeline → Ok.

New Item

Enter an item name

Python-project

Select an item type



Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

Step no:3 Configure page will open → scroll down → Definition: **Pipeline Script from SCM** → choose Git.

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/Reshufowzi/python-jenkins-demo.git

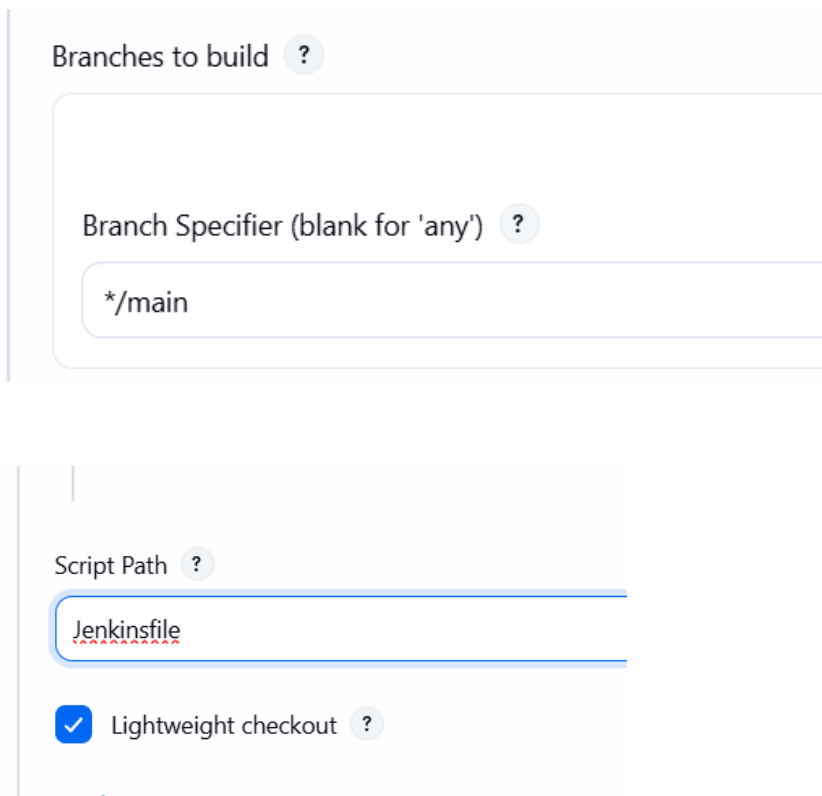
It will ask for repo url. Go to github and get the url of the java project.

The screenshot shows a GitHub repository page for 'Reshufowzi' with the file 'Update app.py' selected. The file explorer on the left shows a directory structure with 'app' and 'tests' folders, and 'Jenkinsfile' and 'README.md' files. A dropdown menu is open over the repository, showing options for cloning. The 'Local' tab is selected, and the 'Clone' button is highlighted. Below the 'Clone' button, there are three tabs: 'HTTPS', 'SSH', and 'GitHub CLI'. The 'HTTPS' tab is active, and the URL 'https://github.com/Reshufowzi/python-jenkins-demo.git' is displayed. Below the URL, there is a button to copy the URL. Below the URL, there is a text 'Clone using the web URL.' Below the text, there is a button to 'Open with GitHub Desktop' and a button to 'Download ZIP'.

Ensure we are entering the correct branch of the Jenkinsfile and the project code.

In script path enter the correct path of the Jenkinsfile in the repo. Globally it is advised to use Jenkinsfile as the name.

Best practice is to keep the file in the root level. If it is inside, we have to provide the absolute path, which is kind of complex.



The screenshot shows the Jenkins configuration interface. Under the 'Branches to build' section, the 'Branch Specifier (blank for \'any\')' field is set to `*/main`. Below this, the 'Script Path' field is set to `Jenkinsfile`. The 'Lightweight checkout' checkbox is checked.

Step no:4 Here we are going to push the respective image to the docker hub. Hence we need docker to installed in our Jenkins machine.

Update the instance and install docker.

sudo su

sudo apt update

sudo apt install docker.io -y

```
Hit:7 http://security.ubuntu.com/ubuntu resolute-security InRelease
39 packages can be upgraded. Run 'apt list --upgradable' to see them.
root@ip-172-31-14-119:/home# sudo apt install docker.io -y
Installing:
  docker.io
```

Start and enable the docker

sudo systemctl start docker

sudo systemctl enable docker

```
root@ip-172-31-14-119:/home# sudo systemctl start docker
root@ip-172-31-14-119:/home# sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service;
   Active: active (running) since Wed 2026-06-03 13:17:15 U
```

```
root@ip-172-31-14-119:/home# sudo systemctl enable docker
root@ip-172-31-14-119:/home#
```

Add Jenkins to the docker group: **sudo usermod -aG docker Jenkins**

Check the status of docker: **sudo systemctl status docker**

```
root@ip-172-31-14-119:/home# sudo usermod -aG docker jenkins
root@ip-172-31-14-119:/home# sudo systemctl status docker
```

After each installation in the instance we need to restart the Jenkins so that the changes will be reflected on our Jenkins page.

sudo systemctl restart Jenkins

```
root@ip-172-31-14-119:/home# sudo systemctl restart jenkins
root@ip-172-31-14-119:/home#
```

It will ask to sign in again after the installation. Sign in with the user name and password.



Sign in to Jenkins

Username

Password

☐ Keep me signed in

Sign in

Step no:5 Now we have to install the python in the instance.

sudo apt update

sudo apt install python3-venv -y

```
root@ip-172-31-14-119:/home# sudo apt update
Hit:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:3 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease
Ign:4 https://pkg.jenkins.io/debian-stable binary/ InRelease
Hit:5 https://pkg.jenkins.io/debian-stable binary/ Release
Hit:7 http://security.ubuntu.com/ubuntu resolute-security InRelease
39 packages can be upgraded. Run 'apt list --upgradable' to see the upgradeable packages.
root@ip-172-31-14-119:/home# sudo apt install python3-venv -y
Installing:
python3-venv
```

After each app installation in the instance we need to restart the Jenkins so that the changes will be reflected on our Jenkins page. Also the test the python app.

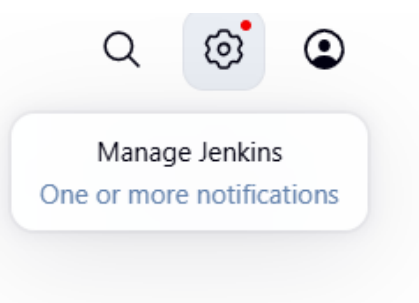
sudo systemctl restart Jenkins

python3 -m venv testenv

```
root@ip-172-31-14-119:/home# sudo systemctl restart jenkins
root@ip-172-31-14-119:/home# python3 -m venv testenv
root@ip-172-31-14-119:/home#
```

Step no:6 As we mentioned we have added the credentials for docker in Jenkins using the credential id, we need to configure that inside the Jenkins machine.

Go to the **manage jenkins**

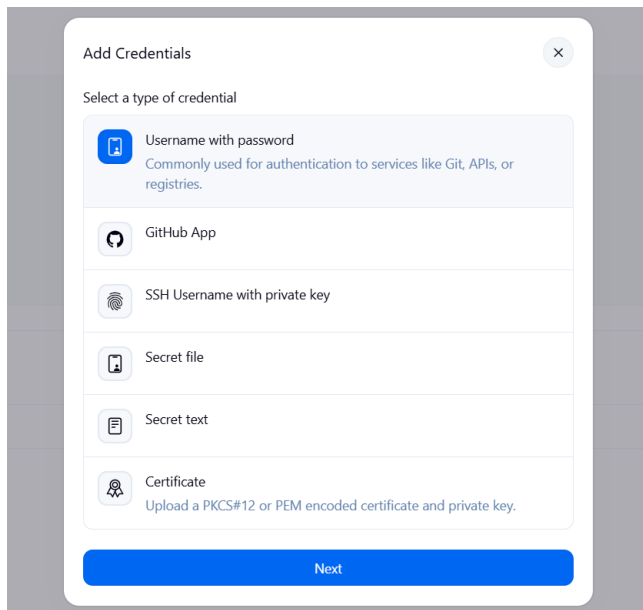


Choose **credentials**

Security

- **Security**
Secure Jenkins; define who is allowed to access/use the system.
- **Credentials**
Configure credentials
- **Users**
Create/delete/modify users that can log in to this Jenkins.

The type as username and password (docker) → next.

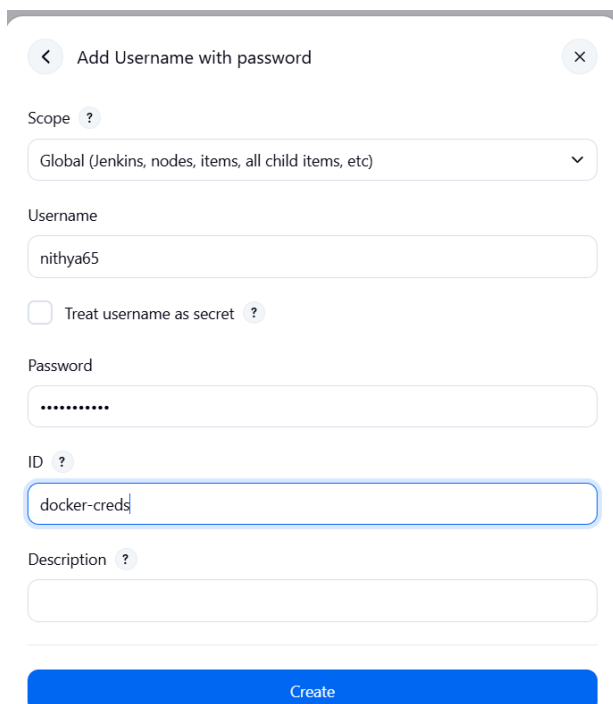


The image shows a modal dialog titled "Add Credentials" with a close button (X) in the top right corner. Below the title, it says "Select a type of credential". There is a list of seven credential types, each with an icon and a description:

- Username with password**: Commonly used for authentication to services like Git, APIs, or registries. (This option is highlighted with a blue background)
- GitHub App**
- SSH Username with private key**
- Secret file**
- Secret text**
- Certificate**: Upload a PKCS#12 or PEM encoded certificate and private key.

At the bottom of the dialog is a blue button labeled "Next".

Add our dockerhub username and our password. It is important to correctly mention the credential id for reference in the jenkinsfile. It will refer it login the account to push the image. Here it is “**docker-creds**”

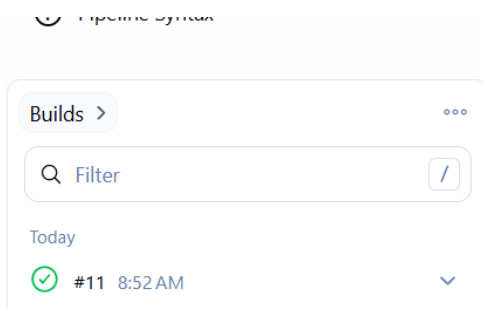


The image shows a form titled "Add Username with password" with a back arrow and a close button (X) in the top left corner. The form contains the following fields:

- Scope**: A dropdown menu with a question mark icon, currently set to "Global (Jenkins, nodes, items, all child items, etc)".
- Username**: A text input field containing "nithya65".
- Treat username as secret**: A checkbox that is currently unchecked, with a question mark icon.
- Password**: A password input field with masked characters (dots).
- ID**: A text input field with a question mark icon, containing "docker-creds".
- Description**: A text input field with a question mark icon, currently empty.

At the bottom of the form is a blue button labeled "Create".

Step no:7 Now go the Jenkins project page → Choose **build now** → once the build has been executed → Go to the **console output**.



Once the build is done, login to dockerhub we can see the image has been pushed in to it.

```
Successfully built a9c5604f824f
Successfully tagged nithya65/python-project:11
+ docker tag nithya65/python-project:11 nithya65/python-project:latest
+ docker push nithya65/python-project:11
The push refers to repository [docker.io/nithya65/python-project]
-
+ docker push nithya65/python-project:latest
The push refers to repository [docker.io/nithya65/python-project]
-
[Pipeline] echo
[Pipeline] }
[Pipeline] }
[Pipeline] }
[Pipeline] }
[Pipeline] }
[Pipeline] }
```

Now we can bring the image to our ec2 console and run it as a container to see the application.

The image tag will be the build number attached to it. Because that is the format we have provided.

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
nithya65/python-project:11	a9c5604f824f	224MB	55.1MB	
nithya65/python-project:latest	a9c5604f824f	224MB	55.1MB	

[Repositories](#) / [python-project](#) / [General](#)

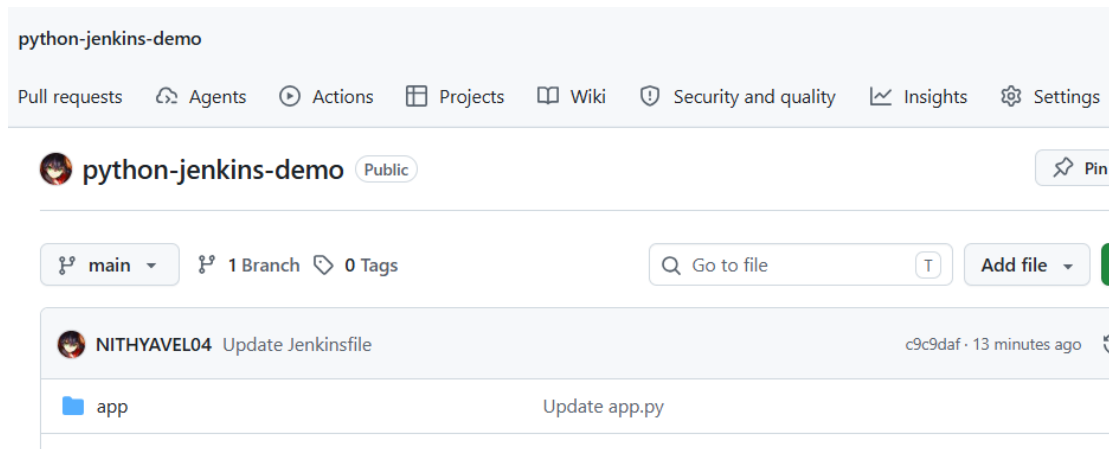
nithya65/python-project

Last pushed 9 minutes ago • ☆0 • ⬇12

TAG			
11			
Last pushed 9 minutes by nithya65			
docker pull nithya65/python-project:11			
Digest	OS/ARCH	Last pull	Compressed size
a9c5604f824f	linux/amd64	less than 1 day	52.57 MB

Hands on: Webhook setup

Step no:1 The webhook is a connectivity tool for Jenkins and github. It will trigger if any changes in Jenkins made in the github.



Step no:2 Go the project repo → settings → add webhook → add the Jenkins url /github-webhook/

Webhooks / Add webhook

We'll send a POST request to the URL below with details of any subscribed event format you'd like to receive (JSON, x-www-form-urlencoded, etc). More [information documentation](#).

Payload URL *

Content type *

Secret

Content type → application-json → Choose: Send me everything → add webhook

Which events would you like to trigger this webhook?

- ☐ Just the push event.
- ☒ Send me everything.

Webhooks

Webhooks allow external services to be notified when certain a POST request to each of the URLs you provide. [Learn more](#)

✓ <http://13.203.201.96:8080/github-w...> (all events)
Last delivery was successful.

Step no:3 Now go to the Jenkins respective project → configure → gitHub webhooks trigger.

Triggers

Set up automated actions that start your build based on specific events

- ☐ Build after other projects are built ?
- ☐ Build periodically ?
- ☒ GitHub hook trigger for GITScm polling ?

Step no:4 Make any changes in respective project and make commit → we can see in Jenkins the build has been automatically executed.

Q Filter

Today

✓

#12 9:14 AM

✗

▼